

Valeur **Logiques** du M1 Info
Partie Logique du premier ordre
Énoncé du TP 1

1 Introduction

On souhaite vérifier des formules logiques sur des graphes non orientés. Récupérez sur ecampus le programme python `graphe_template.py` qui devra être complété. Il contient une classe `Graphe` dans laquelle un graphe est représenté comme une liste python de liste de voisins. Vous définirez une méthode de la classe `Graphe` pour chaque formule, cette méthode renverra `vrai` si la formule est vraie et `faux` si elle est fausse. La classe `Graphe` possède un constructeur qui détermine le nombre de sommets n et une méthode `ajoute_plusieurs_aretes(self,m)` qui permet de contruire un graphe avec m arêtes ajoutées aléatoirement. Lorsque vous avez vérifié qu'une méthode vérifiant une formule est correcte, testez-la en construisant des graphes en fixant le nombre de sommets et en faisant varier le nombre d'arêtes. Essayez de voir pour quelles valeurs elle renvoie `vrai` et pour quelles valeurs elle renvoie `faux`.

2 Énoncés du premier ordre

Question 1. Codez la méthode `phi1` qui vérifie l'énoncé du premier ordre suivant

$$\varphi_1 = \forall x \left(\left(\exists y E(x, y) \right) \wedge \left(\exists y \neg E(x, y) \right) \right).$$

Question 2. Un triangle dans un graphe non orienté est un triplet de sommets distincts (a, b, c) vérifiant

$$(a, b) \in E \text{ et } (b, c) \in E \text{ et } (a, c) \in E.$$

- a) Donnez un énoncé du premier ordre φ_2 exprimant la propriété que tout sommet a appartient à un triangle.
- b) Codez la méthode `phi2` qui vérifie φ_2 .

Question 3. On considère l'énoncé du premier ordre φ_3 suivant

$$\varphi_3 = \left(\forall x \forall y \left(x \neq y \rightarrow \left(E(x, y) \vee \exists z \left(E(x, z) \wedge E(z, y) \right) \right) \right) \right) \wedge \left(\exists x \exists y \neg E(x, y) \right).$$

- a) Quelle propriété est exprimée par l'énoncé φ_3 ?
- b) Codez la méthode `phi3` qui vérifie φ_3 .

3 Clôture transitive et connexité (partie plus difficile)

Nous souhaitons vérifier qu'un graphe non orienté G est connexe. Pour cela, nous allons construire la clôture transitive du graphe. Celle ci sera stockée dans un nouveau graphe `self.cloture`.

Question 4. On rappelle que la clôture transitive de G est l'unique relation binaire S vérifiant l'énoncé suivant

$$\varphi_4 = \forall x \forall y \left(E(x, y) \vee (\exists z E(x, z) \wedge S(z, y)) \right) \rightarrow S(x, y).$$

On commence par faire une copie du graphe G (première partie de φ_4).

Complétez la méthode `iteration(self)` avec une **triple boucle** afin de parcourir tous les sommets a du graphe pour essayer d'ajouter un couple (a, b) à la relation S (seconde partie de φ_4). Vous renverrez `vrai` lorsqu'au moins un couple a été ajouté à S .

La méthode `cloture_transitive(self)` appelle la méthode `iteration` jusqu'à ce que l'on ne puisse plus ajouter de couple (a, b) à S .

Question 5.

- a) Comment vérifiant-on que le graphe G est connexe à partir de sa clôture transitive ?
- b) Complétez la méthode `est_connexe` afin de vérifier que G est connexe.